Sarika pujari
AMB2223

Python – Advanced Activity Handouts (Project & Problem-Solving Focus)

Activity A1 – From Idea to Project Specification

Title: Define Your Mini-Project Clearly

Goal
Turn a rough idea into a precise, buildable project description.

Instructions
Pick one idea (examples: simple hospital appointment system, dataset analyzer, quiz app, log analyzer, file organizer).

Part A – One-page project brief

1. Project title:

log analyzer

2. Problem you want to solve (2–3 short sentences):

System administration often need to quality detect errors suspicious activity or unusual patterns in large big files

3. Target users (who will actually use this?):

* system administration, * Develops engineers

4. Inputs (what data / actions will the program receive?):
   - log file path
   - optional filters
5. Outputs (what will the user see / get?):
   - summary statistics
   - Filtered big entries
6. Constraints or assumptions (keep it realistic):
   - logs are text based
   - Runs locally

Part B – Core features vs "nice-to-have"

1. List minimum features needed for a working version (MVP):
   - load log file
   - Filter by log level
   - count occurence per level
2. List optional features (only if time allows):
   - Real - time streaming mode
   - suspicious pattern, detection
3. Final question:
   - If you had only 1 week, which ONE feature would you absolutely keep? Why?

I would keep log filtering and counting because that delivers immediate practical value and forms the foundation for all advanced features

Activity A2 – System Decomposition (Modules & Responsibilities)

Title: Break Project into Modules

Goal
Learn to split a project into logical modules and functions.

Instructions
Use your project from Activity A1.

Part A – High-level modules

1. List 3–6 modules (e.g., data_input, processing, reporting, ui):
   - Module 1: data - entry
   - Module 2: file - management
   - Module 3: Search - and - query
   - Module 4: report - generation
2. For each module, write its main responsibility in one sentence:
   - Module 1 does: Handles user input to collect student details like name, ID, and grades.
   - Module 2 does: Manages reading from and writing data to external text or binary files.
   - Module 3 does: Locates specific student records based on search criteria like ID
   - Module 4 does: Formats and displays student data as a clean readable summary or transcript

Part B – Functions inside modules

For one key module (choose the most important):

1. Module name: file - management
2. Functions planned:
   - Function 1 name: Save - record
     - Input(s): Student data object/dictionary, filename,
     - Output: Success confirmation
   - Function 2 name: load - all - records
     - Input(s): File name
     - Output: A list containing all student records found in the file
   - Function 3 name: update - record
     - Input(s): Student ID, new data, filename
     - Output: The only updated file status.
3. Draw or briefly describe how data flows between these functions:

The data flow within the file - management module centers around a persistent storage file (like a .txt or .json file. Here is how the functions interact

Activity A3 – Designing Data Structures

Title: Choosing the Right Data Structures

Goal
Practice selecting lists, dicts, sets, tuples, or custom classes.

Instructions
Consider your project's core data (e.g., patient records, quiz questions, log entries, transactions).

Part A – Describe one key entity

1. Entity name (e.g., "Appointment", "Record", "Transaction"):
   Student

2. For this entity, list required fields:
   - Field 1: Student (type: int/str/float/...)
   - Field 2: name (type: str)
   - Field 3: gpa (type: float)
   - Field 4: enrolled (type: bool)

Part B – Pick implementation

Choose one representation and justify:

1. Option A – Dictionary:

python

```
item = {
  "field1": student_id (type: int)
  "field2": name (type: str)
  ...
}
```

2. Option B – Class:

python

```
class Item:
  def __init__(self, field1, field2, ...):
    self.field1 = field1
    self.field2 = field2
  ...
```

3. Which option will you use, and why?
   I will use option B (class): while dictionaries are great for quick data storage, a class is better for a "student"

4. Write a short example of creating and using this structure:

python

```
# Example creation and simple usage
```

Activity A4 – Planning Tests Before Coding

Title: Think in Test Cases (TDD Lite)

Goal
Build habit of designing tests before writing code.

Instructions
Choose one non-trivial function from your project (e.g., fee calculation, filter logic, scoring rule).

1. Function name: calculate_late_fee
2. In simple words, what should this function do?

The function should calculate a penalty fee based on how many days a payment is overdue

3. List at least 5 test cases (inputs and expected outputs):

Test #	Input(s)	Expected output
1	0	0
2	1	5
3	10	50
4	-3 (early payment)	0
5	0.5 (partial day)	2.5 (or 5 if rounding)

4. Now write a small test harness:

```python
def my_func(...):
    # TODO: implement
    pass


# Tests
print("Test 1:", my_func(...), "expected:", ...)
print("Test 2:", my_func(...), "expected:", ...)
```

5. Reflection:

- Did writing tests first change how you thought about the function? How?

Yes, By writing the tests first, I realized I needed to handle "edge cases" like what happens if some one provides a negative number for days. Without the test cases, I might have accidentally written code that returned a negative fee, which wouldn't make sense in a real-world scenario. It made the logic much clearer before I even started typing the if statement.

Activity A5 – Error Handling & Robustness in a Project

Title: Make Your Project Hard to Break

Goal
Identify and protect weak points with validation and exception handling.

Instructions
Look at your project idea and identify three likely failure points (e.g., wrong file path, invalid input, network issue).

1. Failure Point 1: Invalid data|time format
   - How to detect it? try converting formed
   - How to handle it politely for the user? Ask user to re-enter
2. Failure Point 2: Duplicate booking
   - Detect: Validation function
   - Handle: Show warning message
3. Failure Point 3: Empty input
   - Detect: Check string length
   - Handle: prompt again

Part B – Add try/except around one risky area

Example skeleton:

python

```
file_path = input("Enter file path: ")


try:
  with open(file_path, "r") as f:
    data = f.read()
  print("File loaded successfully.")
except FileNotFoundError:
  print("File not found. Please check the path.")
except PermissionError:
  print("Permission denied. Try a different file.")
```

Adapt this pattern to your project and paste your modified code below:

python

 # Your robust code:

```
doctor = input("enter doctor name:
try:
    if doctor == " ":
        raise value error("Doctor name
                connot be emp
    print("Doctor selected:" doctor)
except value error as e:
    print("Error:" e)
```

Activity A6 – Refactoring for Clarity

Title: Improve Existing Code (Same Behaviour, Better Design)

Goal
Practice cleaning up messy, working code into clear, modular code.

Instructions
Take one working script (from your own work or given by teacher).
It should be correct but long, repetitive, or hard to read.

1. Paste or summarize original code name / purpose:

Single long function handling everything

2. Identify at least three problems (long function, repetition, confusing names, etc.):
   - Problem 1: long function
   - Problem 2: Repeated code
   - Problem 3: Poor variable names
3. Plan refactoring steps (high level):
   - Step 1: Split into smaller functions
   - Step 2: Improve naming
   - Step 3: Add validation function
4. After refactoring, write or paste your improved version here:

python

 # Refactored code:

5. Reflection:
   - What became easier to understand after refactoring?

```
def get_patient_details():
    name = input("Enter patient name:")
    return name
def main():
    name = get_patient_details()
    print("patient", name)
main()
```

Day A7 – Building a Command-Line Interface (CLI)

Title: Make Your Project Usable from Terminal

Goal

Learn to take arguments and options from the command line.

Instructions

Turn some functionality of your project into a small CLI tool.

1. Define a simple CLI command:
   Example: python hospital.py add --name, Nandsha --doctor Mehta --date 20-2-26 --time 10AM
   - python student_tool.py add --name Alice --score 85
2. What should the tool do in ONE sentence?
   the tool allows the uses to book & manage hospital appointments through command-line inputs in a simple way
3. Sketch CLI arguments:

Argument / Option	Meaning
--name	patient name
--age	patient age
--doctor	Doctor Name

4. Implement with argparse (skeleton):

python

```
import argparse

parser = argparse.ArgumentParser(description="Describe your tool here")
parser.add_argument("--name", type=str, help="Student name")
parser.add_argument("--score", type=int, help="Student score")
args = parser.parse_args()

# Use args.name, args.score here
print("Name:", args.name)
print("Score:", args.score)
```

("Hospital Appointment System")

parser.add_argument("action", type=str, help="Action to perform: add/view")
parser.add_argument("--name", type=str, help="pat
parser.add_argument("--age", type=int, help="patient age")
parser.add_argument("--doctor", type=str, help="Dr. name
parser.add_argument("--date", type=str, help="App
parser.add_argument("--time", type=str, help="App

5. Adapt the skeleton to your project and note example commands at the bottom of your file.

```
args = parser.parse_age()
If args.action = "add":
    print("Adding appointment---")
    print("patient", args.name)
    print("doctor", args.doctor)
    print("date", org.date)
    print("time:", org.time)
elif args.action == "view"
    print("viewing appointments for doctor:", org.date)
```

Activity A8 – Mini Capstone: End-to-End Project Plan & Review

Title: From Zero to Demo

Goal
Produce a realistic plan and self-review for your project.

Part A – Milestones and Timeline

1. List 4–6 milestones (each should be testable):
   - Milestone 1: Finalize project idea & requirements
   - Milestone 2: Design data structure
   - Milestone 3: Implement core feature
   - Milestone 4: Add input validation & error handling
   - Milestone 5: _____

2. For each milestone, define:

Milestone	Concrete Deliverable	Due Date
1	project plan document	Day 1
2	class structure code	Day 2
3	working core functions	Day 3

Part B – Risk & Mitigation

1. Identify 2–3 biggest risks (technical or time):
   - Risk 1: Time constraint
     - Plan B if this fails: focus only on basic features
   - Risk 2: Logical errors in booking system
     - Plan B if this fails: use test cases before coding & decoding step-by-step
   - Risk 3: Improper input handling
     - Plan B if this fails: Add simple validation

Part C – Final reflection after project

(Do this at the end.)

1. When I got stuck, what strategies actually helped me get unstuck?

Breaking the problem into smaller parts & testing step-by-step helped me

2. One thing I now believe about myself as a problem-solver:

I can solve problems with proper planning & patience

3. If I start a new project tomorrow, what will I do differently from day 1?

I will plan the structure & test cases before starting coding